MAP Open Intrusion Interface

Base Specification

BOSCH

en Technical Description

Table of Contents

1 Definitions, Acronyms and Abbreviations

OII	Open Intrusion Interface
REST	Representational State Transfer
HTTP	Hypertext Transfer Protocol
TCP	Transmission Control Protocol
TLS	Transport Layer Security
IP	Internet Protocol
MAP	Modular Alarm Platform
Etag	Entity Tag
URI	Uniform Resource Identifier

2 Introduction

2.1 Purpose

This document describes the communication technology and concept for the access to and control of the MAP5000 panel. The specified solution will enable the MAP5000 to communicate to multiple clients such as management systems or smart phones in parallel.

The interface is following a RESTful design pattern, where the MAP5000 is offering resources as a server that can be inspected and modified by a client using the HTTP protocol.

Standard security mechanisms TLS, HTTP digest are used to achieve secure communication.

2.2 Scope

The goal of the OII is to allow an easy access to the MAP5000 which does not require dedicated drivers or proprietary technologies to interact with the panel. Thus, widely used, standard technologies are employed wherever possible, to simplify usage of the OII.

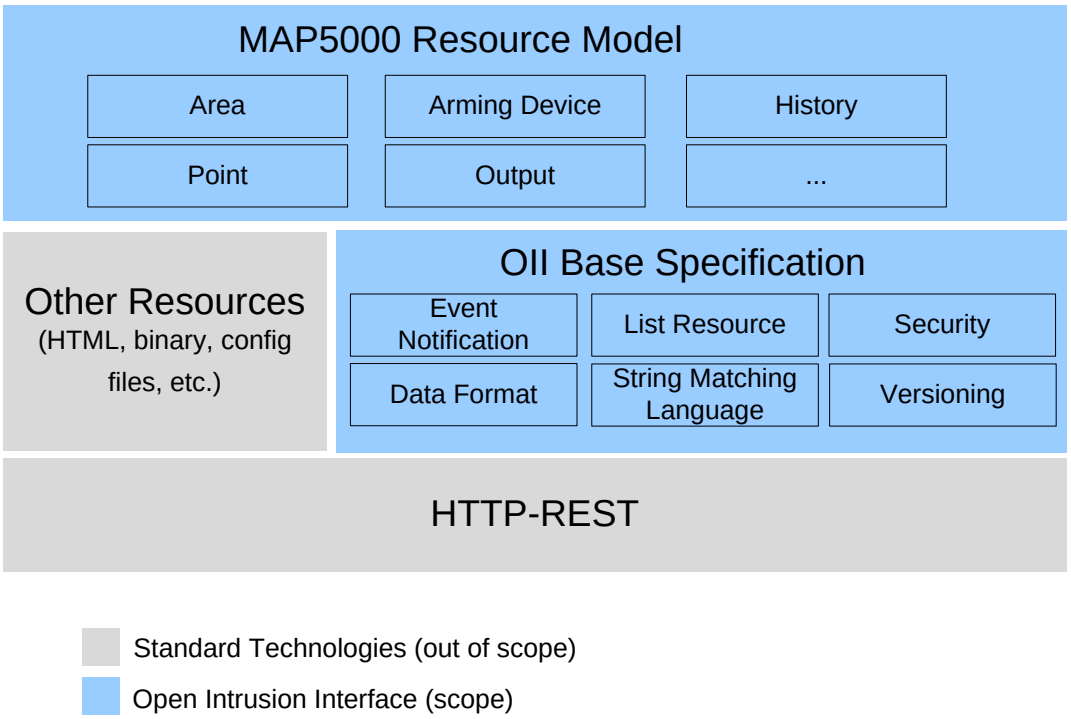


Fig. 1 OII Specification Overview

HTTP-REST
As its basis, the OII follows a RESTful design using HTTP as the communication technology. HTTP-REST already provides a comprehensive set of functionality for communication in distributed systems. In context of the OII, the following features are used in particular:

- HTTP 1.1 Protocol
- Verbs GET, PUT, POST, DELETE
- HTTP Response Codes
- HTTP User Authentication (HTTP digest)
- HTTPS based secure communication

OII Base Specification
In order to provide a unified access to a variety of functions of the MAP, the OII base specification adds additional functionality and rules on top of REST.

In particular, the OII base specification defines:

- A unified format to represent information of the MAP based on JSON
- A versioning approach for the interface and its components to ensure forward and backward compatibility
- An event notification mechanism for communication of spontaneous state changes and events
- Batching capabilities for efficient communication and control of multiple resources

OII Resource Model
On top of the OII base specification, the MAP Resource Model defines in detail which resources are provided (e.g. area, point, keypad), which information is contained in a resource and how commands can be issued. Wherever possible the MAP Resources will follow the OII base specification. In some cases, the MAP Resource Model may also use other types of resources to accommodate special functionality e.g. for configuration export or firmware update.

2.3 Example

In order to show case the basic functionality of the OII, we are going to look at a simplified example to inspect an area status and arm the area.

An area itself is modelled as a resource and therefore its status can be easily inspected using an HTTP GET to its address. In the context of the OII, there is a one-to-one relationship between the area's SIID (as it is shown in RPS) and its URL. In our example, the area has the SIID 1.1.Area.2.20. Its corresponding URL is /1.1.Area.2.20. Executing a GET on "https://<panelIPAddressAndPort>/1.1.Area.2.20" will return:

```
{
    "@type": "IN.area.1",
    "@self":"/ 1.1.Area.2.20",
    "armed": false
}
```

Fig. 2 Example Area Resource

Thus, we can simply inspect the status of the area and can deduct that it is not armed. Please note that this is only a simplified example and that the actual resource will contain more information.

As a second step, we now want to issue an arming command to this area. In context of the MAP, arming is done by issuing a POST request to the area. Thus, we send a POST to the URL "https://<panelIPAddressAndPort>/1.1.Area.2.20" that has the following content in the request body:

```
{
    "@cmd": "ARM"
}
```

Fig. 3 Example Request Body

Considering that the request has been accepted and the arming has been finished successfully, the next GET request on the area will return:

```
{
      “@type”: “IN.area.1”,
      “@self”:”/ 1.1.Area.2.20”,
      “armed”: true,
}
```

Fig. 4 Example Area Resource

3 Basic Definitions

3.1 Data Format

Resources compliant to the OII base specification will be encoded using JSON, as it provides a suitable trade-off between data size and a well structured, human readable data format. Libraries for JSON are available for most platforms and programming languages.

JSON itself basically defines the notion of objects which consist of name-value pairs. A value itself can be an object, array, number, string, true, false or null. This allows the design of simple key-value based data structures like in the example in Fig. 2, as well as complex structures with object hierarchies like the one below.

The specific data structure of a dedicated resource is defined in scope of the MAP5000 Resource Model. Following the specification, the JSON object may contain whitespaces and tabs in order to make it easier for humans to read. However, the data provided by the MAP will not contain whitespaces or tabs in order to save bandwidth. It is suggested that clients also remove any unnecessary mark up when sending data to the MAP for the same reason.

```
{
  "@type": ["IN.areaList.1","OII.list.1"],
  "@self":"/areas",
  "list":[
    {
      "@type": "IN.area.1",
      "@self":"/1.1.Area.1.1",
      "armed": true
    },
    {
      "@type": "IN.area.1",
      "@self":"1.1.Area.1.2",
      "armed": false
    }
  ]
}
```

Fig. 5 Example of complex JSON Object

In addition, OII clients need to make sure that they support the JSON format, including the particular encoding rules for strings and numbers as defined in the JSON specification.

3.2 Property Names

JSON properties used in the scope of the OII must comply with the following rules (based on [JSTY]):

- Property names should be meaningful names with defined semantics.
- Property names must be camel-cased, ascii strings.
- The first character must be a lowercase letter, an underscore (_) or a dollar sign ($).
- Subsequent characters can be a letter, a digit, an underscore, or a dollar sign.
- Reserved JavaScript keywords should be avoided (A list of reserved JavaScript keywords can be found in [JSTY]).

Three property names and their semantics are specifically reserved:

Property Name	Property Value
@type	A list of resource type identifiers specifying data model and operations available for this resource
@self	A relative link specifying the URL of this resource on the OII server.
@cmd	A string identifier of the command that is to be invoked. This property is used

	in the scope of a POST command to distinguish different commands for the same resource type (e.g. arm and start walk test)

3.3 Resource Types and Versioning

As the OII Interface is the designated interface to the MAP5000 for the next years, it can be expected that the interface itself will evolve and change over time. In order to assure that even clients and servers that support different versions of the OII remain compatible, each resource is annotated with a dedicate resource type. The resource type specifies the object structure of the resource as well as the supported verbs and input/output parameters for the requests.

In order for a client to be informed of the resource type of a given resource, a resource includes the property "@type". The type itself is an array of type-strings. This allows definition of interface inheritance relationships where a resource complies with multiple resource type definitions, meaning that it provides all the associated objects and commands of the associated types.

In order to facilitate versioning of the interface, the type-string structure is as follows:

<namespace>.<name>.<major>[.<minor>]

- **Namespace**: Mandatory element to indicate the overall context the type is derived from. The namespace "OII" is reserved for types defined in the OII base specification. The MAP5000 specific resource types use the namespace "IN", indicating the context of intrusion detection systems.
- **Name**: Mandatory element identifying the general resource type e.g. list, area, point.
- **Major**: Mandatory element defining the major version number. Clients and server that support the same major server number should be generally compatible.
- **Minor**: Optional element indicating minor changes in the type (0, if not specified). The resource is compatible with all prior minor versions, meaning that the same information objects are available and the commands defined for the resource are supported. A minor resource change may for example be the addition of a new object into the resource.

Examples for resource types can be found in Fig. 6

```
 {
   "@type": ["IN.areaList.1","OII.list.1"]
}
```

```
{
   "@type": ["IN.area.1.24"]
}
```

Fig. 6 Type Examples

3.4 Linking

Sometimes resources are included in other resources (e.g. in lists or events). In these cases, it is important that a client can distinguish where the JSON object originates from. To this end, each OII resource contains a JSON object "@self" which contains a link to itself.

The value of "@self" is the relative path of the resource URL, starting at the server's root e.g. "@self":"/1.1.Area.1.2" or "@self":"/description". The client can use this link to directly interact with the individual resource.

3.5 Minimal Resource Example

Following the above definitions, a minimal OII resource only includes the mandatory JSON properties "@type" and "@self" and supports a GET.

```
{
   "@type": ["ex.sample.1"],
   "@self":"/my/u/r/l"
}
```

Fig. 7 Example Resource

3.6 Timestamps

All timestamps that are used in the context of the OII are compliant to the "Date and Time on the Internet: Timestamps" [RFC3339] which is based on the ISO 8601 standard. It depends on the specific resource whether the precision of the timestamp is in seconds (mandatory) or in milliseconds (optional).

Examples for valid timestamps are:

2014-10-29T09:50:50+00:00
2014-10-29T09:50:50Z
2014-10-29T13:25:22.357-07:00

In some cases the resources will not provide time information. In these cases an ISO 8601 string shall be provided only containing the date information.

Examples for valid date strings are:

2014-10-29
2000-01-01
2008-05-11

3.7 Connection Handling

A client may use one or multiple TCP connection to communicate with the MAP. The MAP will follow the HTTP header information from the client, whether a TCP connection shall be closed or kept open after a HTTP response is sent. Thus, a client may reuse the TCP connection for multiple HTTP requests and thereby save time and bandwidth as TCP and TLS handshake are only required once.

The MAP will close inactive connections after 120 seconds to prevent an unavailable client from hogging system resources. In case the client sends requests regularly, the connection will be kept open.

3.8 Command Execution

Commands that are sent to the system over OII will usually be forwarded to the system for processing and a response is directly provided to the client with a 202 “Accepted” return code. From this follows, that a client cannot assume that the command execution has already succeeded with the execution, as the system may take time to finally take all necessary steps as requested (e.g. arming 500 areas can take several seconds but the OII request will return as soon as the arming process has been started). This also means that it is not guaranteed that the command will be executed as requested. For example, arming of 500 areas may take several seconds to succeed. It may happen that during that time an area becomes not ready to arm, so that the arming of that area does not succeed. As a consequence only 499 areas will be armed eventually. Therefore, a client needs to check for resource state changes of the resources that it wants to control to verify whether command execution succeeded or not. The client has to retry if the required status change (e.g. all 500 areas are “armed”) does not happen.

Thus, a client that requires sequential command execution needs to validate the successful execution of the command using the resource state before the next command is sent.

4 List Resource

In many scenarios clients want to access multiple resources with a single request. For example, a client may be interested in the status of all areas in the panel. In case the panel is configured with 20 areas, this would mean that the client needs to issue 20 requests (one for each area) to get the required status information. In order to save bandwidth and communication effort, so called list resources are defined that allow retrieval of information of multiple resources with a single request.

As access to multiple resources is a general feature, we define the generic resource type “OII.list.1”. A resource of type “OII.list.1” contains an object with the name “OII.list.1”. The “OII.list.1” object is an array which contains a number of OII resources (see Fig. 8).

Depending on the situation, a client may want to access all or only some of the resources of the list e.g. get status of 10 areas and not all 20. In order to allow a fine grained access to the resource list, filters can be defined. In the response to a filter request, only the resources will be included in the “list” object, which match these filters. Thus, filters can be used to optimize the amount of data that is communicated.

```
{
  “@type”: [“IN.areaList.1”,“OII.list.1”],
  “@self”:”/areas”,
  “list”:[
    {
      “@type”: “IN.area.1”,
      “@self”:”/1.1.ControlPanelArea.1.1”,
      “armed”: true
    },
    {
      “@type”: “IN.area.1”,
      “@self:”/1.1.Area.2.1”,
      “armed”: false
    },
    {
      “@type”: “IN.area.1”,
      “@self”:”/1.1.Area.2.2”,
      “armed”: false
    }
  ]
}
```

Fig. 8 Simple List Resource Example

Filters are specified in the query part of the URL. The following query parameters are supported by a list:

- **URL (url)** A comma separated list of resource URLs to be included in the response to a GET request e.g. https://panel/areas?url=/1.1.Area.2.5,/1.1.Area.2.7. In addition, the string matching language defined in Chap 6 is used. This allows referencing of multiple resource URLs with a short expression instead of listing all requested resource URLs e.g. https://panel/areas?url=/1.1.Area.2.(5-15), https://panel/areas?url=/1.1.Area.2.*
- **Atomic command (atomic)** A parameter without value that specifies that a command executed on a list shall be executed in an atomic way (e.g. https://panel/areas?atomic&url=/1.1.Area.2.(5-15)). This means that the command shall

only be executed if all resources selected from the list can execute the command. For example, if an arming command is send to a list of areas with the atomic flag, than the command will only be executed if all areas are ready to arm. If one area is not ready to arm, the command will be executed on none of the areas. A 409 “Conflict” response will be provided with the list of resource URLs which prevented the execution of the command. The list will be presented as a comma separated value string (e.g. /1.1.Area.2.1,/1.1.Area.2.2,/1.1.Area.2.5) as content of the error response. Content type of the response will be text/plain. Please note that the atomic flag prevents execution of a command based on the status information of the resources at the time when the request is received. After the command has been started it still may happen that the command execution fails (e.g. device goes off normal before the arming command of the area is processed completely).

In the absence of the atomic parameter the command is attempted to be executed on all list entries where it is possible. In this case a 202 “Accepted” response will be provided. The client will need to check on the individual resources whether the command has really succeeded and the related resource property has changed as expected. In case the resource does not change, it may have happened that the internal rules of the MAP5000 have prevented the command to be executed. In case the command cannot be executed on any element of the list, the 409 “Conflict” error response will be provided. The content of the response will be “Not executed” as a plain text.

Fig. 9 depicts the response to a GET request to the list resource of Fig. 8 when using the following filter https://panel/areas?url=/1.1.Area.2.*.


```
{
  “@type”: [“IN.areaList.1”,“OII.list.1”],
  “@self”:”/areas”,
  “list”:[
    {
      “@type”: “IN.area.1”,
      “@self”:”/1.1.Area.2.1”,
      “armed”: false
    },
    {
      “@type”: “IN.area.1”,
      “@self”:”/1.1.Area.2.2”,
      “armed”: false
    }
  ]
}
```


Fig. 9 List Resource Response with Filter

A generic OII list resource only supports GET with the defined query parameters. PUT, POST and DELETE are not supported by the generic OII.list.1 type. In case the query parameters do not comply with the defined format, an error “400 Bad Request” will be returned.

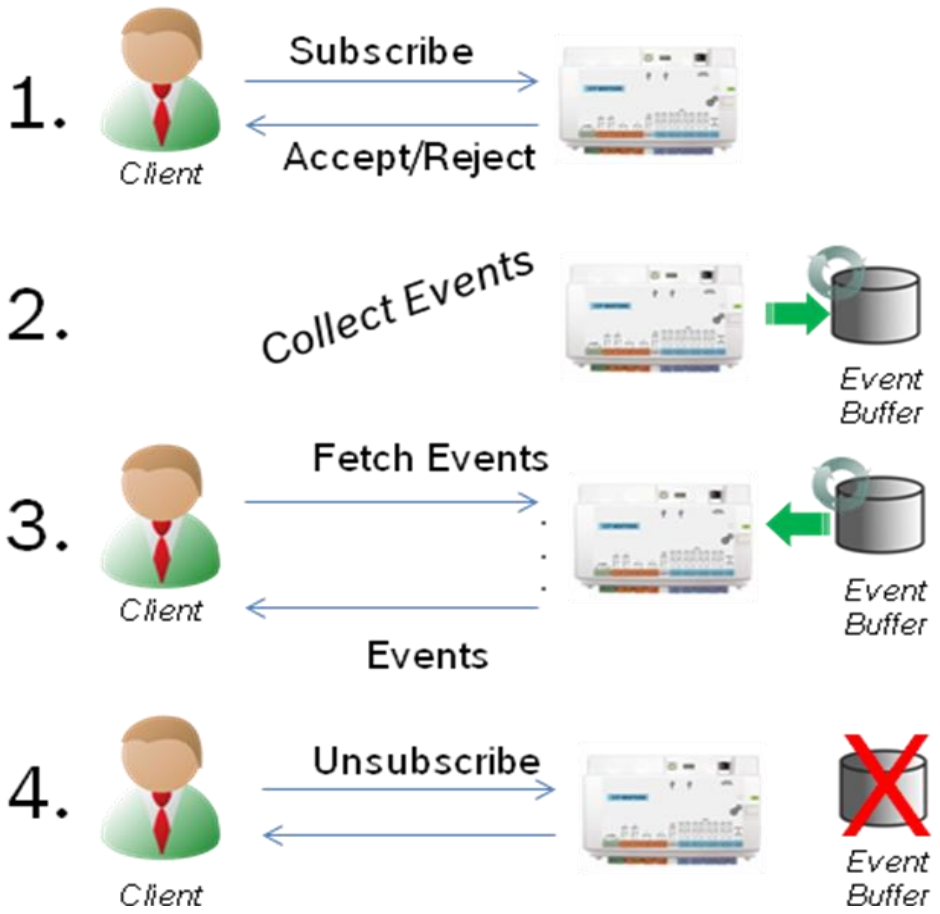
Specific list resources can be defined that inherit from the generic list type and extend it with support for other verbs and commands. For example, the resource type “IN.areaList.1” allows arming of multiple areas by sending a POST to the list of areas. The specific list resources can also reuse the defined query parameters in the context of other verbs, e.g. to execute an arming on a subset of all areas.
The list resource will respond with a 409 (“Conflict”) if it doesn’t support the command.

5 Event Notification

5.1 General Concept

The OII includes an event notification mechanism that allows spontaneous transmission of state changes (e.g. closing of a door contact) and occurrence of incidents (e.g. intrusion alarms). The notification mechanism uses a publish-subscribe pattern, where a client has to register with the MAP to specify which events it is interested in. This allows to custom tailor the events collected for each individual client. For example, a client may subscribe only for incidents in the system. In this case, it will only be notified about alarms and troubles but will not be notified about state changes of peripherals, significantly reducing the amount of events to be communicated.

Once a subscription has been successfully created, the MAP will internally store the events for the client in a ring buffer with a defined size. It is the task of the client to fetch the events by issuing a request to a subscription specific URL.



Event delivery

The delivery of the events is done using a so called long polling mechanism. This means that an incoming request to fetch events will not return an answer until either a sufficient amount of events is available or a timer expires. A client specifies for each request how much events shall be included in the response as a minimum (minEvents), as a maximum (maxEvents) and how long the call is allowed to block at most (maxTime).

In general, a client needs to fetch its events periodically to make sure that all events are received. The MAP will store only a limited amount of events for the client. In case events are not fetched in time, the oldest events in the buffer are overridden (ring buffer). A client can inspect the event id to recognize a buffer overflow. Even after a buffer overflow, MAP will continue to collect events. It is the choice of a client if an overflow is acceptable or not. If not, a client can unsubscribe to free the buffer and create a new subscription.

Lease time

A client also needs to ensure to poll for new events periodically, as the MAP will delete subscriptions that have been inactive for a given time (lease time negotiated during subscription). In cases where a client is not able to continuously poll events e.g. due to overload conditions, a client can fetch events with maxEvents set to 0. Such a request will be interpreted as subscription renewal but no events will be sent to the client.

Event structure

The system will create events in case of creation of a new resource, a state change of an existing resource or on deletion of a resource. Each event will have a unique ID, a timestamp which marks the time of occurrence, a type (created/changed/deleted) and the complete resource representation. In addition, a list of property keys is included in a state change event to indicate the properties that have changed in the resource. Fig. 10 depicts an example event.

The JSON object structure of an event is as follows:

- **id**: An ID that uniquely identifies the event. The event id has the following structure
  <PanelEventID>.<SubscriptionSequenceNumber>
  Panel Event ID: Panelwide, unique ID for this event. A client shall treat this ID as an opaque string and should not assume any particular semantics of the panel ID value. The panel event ID is independent of the individual subscription and unique even over a power cycle of the MAP.
  SubscriptionSequenceNumber: An integer that identifies an event in the context of the subscription. The sequence number will be incremented for each event that is stored for a specific subscription (starting with 0). A client can inspect the sequence number of an event to deduct whether events have been lost i.e. panel has overridden an event as the ring buffer is full. The number ranges from 0 to 65535. In case this range is exceeded, the number is reset to 0 and continued to be incremented from then on. From this follows that a client needs to be prepared to expect a sequence number of 0 as a valid successor to 65535.
- **time**: Local date time in millisecond precision compliant with the OII timestamp definitions (e.g. 2014-10-29T13:25:22.357-07:00).
- **type**: A general classification of the event. Possible values are CHANGED (a state change in a resource has occurred), CREATED (a resource has been created), DELETED (a resource has been deleted).
- **props**: A list of property keys whose value has changed. The list contains at least a single entry for an event of type CHANGED. In all other cases the list will be empty.
- **evt**: The resource representation of the resource that triggered the event. Please note that also for a DELETED event the last status of the resource will be included.

```
{
        "id": 2345.4,
        "time": "2014-10-29T13:25:22.357-07:00",
        "type": "CHANGED",
        "props": ["armed"],
        "evt": {
           "@type": "IN.area.1",
           "@self": "/1.1.Area.2.1",
           "armed": false
         }
}
```

Fig. 10 Event Example

Resource model
The client uses a central subscription resource (e.g. /sub) to subscribe for events of any resource. If the subscription is accepted, a dedicated resource (e.g. /sub/1) is created which represents the individual subscription. The client uses this resource to manage the subscription (unsubscribe, extend lease) and to fetch events.

In the following, we define the resource model and operations for these resources.

5.2 Subscription List Resource

The subscription list resource allows access to all subscriptions. Furthermore, it is the location where a client can create a subscription.

Overview

OII Type	Supported Operations
OII.subList.1 OII.list.1	Status Retrieval Subscribe

Object Structure

Name	Type	Value Range/Format	Description
@type	Array of strings	["OII.subList.1", "OII.list.1"]	Fixed type identifier
@self	String	URL starting with "/"	Link to the current resource
list	Array	Array of subscription object structures	List of current subscriptions (respecting the access rights of the user)

```
{
  "@type": ["OII.sub.1"],
  "@self": "/sub/1",
  "list": [
     {
       "@type": ["OII.sub.1"],
       "@self": "/sub/1",
       "leaseTime": 600,
       "bufferSize": 50,
       "subscriptions": [
       {
        "urls": [ "*" ],
        "eventType": ["CREATED", "DELETED", "CHANGED" ]
       }
      ]
     },
     {
       "@type": ["OII.sub.1"],
       "@self": "/sub/2",
       "leaseTime": 300,
       "bufferSize": 100
       "subscriptions": [
       {
        "urls": [ "*" ],
        "eventType": ["CREATED", "DELETED", "CHANGED" ]
       }
      ]
     }
  ]
}
```


5.2.1 Status Retrieval

The information about the current subscriptions is provided, as defined in the previous sections.

Request

Verb	*GET*		
Query Parameters	url	Resource ID as specified for OII.list.1	optional

Response (Success)

Return Code	200
Content	Subscription List object structure. The list object will only contain resources which match the defined filters given in the query

	parameters.

Response (Error)

See Chap. 11 Standard Errors.

5.2.2 Subscription

In order to create a subscription, a client has to send a POST request to the subscription list resource. The body of the POST request contains all relevant information about the subscription.

This subscription details are:

- **bufferSize**: Number of events that will be stored by the MAP without overwriting an entry.
- **leaseTime**: Maximum number of seconds between two subscription renewal activities i.e. time between two event fetching requests. Lease time is encoded as JSON number with no fractions supported. Maximum lease time is 600 seconds. Minimum lease time is 10 seconds.
- **subscriptions**: An array of subscriptions containing the following information
  - **eventType**: Array of event types (CHANGED, CREATED, DELETED)
  - **urls**: List of links relative to the server base URL from which events are to be received. The list may contain strings that follow the string matching as defined in Chap. String matching6 to reference multiple resources in a compact way. In case a url is pointing to a list resource, the subscription will be created for each element in the list (not the list itself). The client will also be informed of creation and deletion of resources in that list, in case it is selected as part of the eventType.

In case the MAP accepts the subscription, it will respond with a 201 ("Created"). The response will contain further details about the subscription. The subscription will be created only for those resources to which the client has access rights to.

The information provided in the response is as follows:

- **subscriptionURL** A URL where the client can inspect its subscription details. The URL can also be used to unsubscribe.
- **leaseTime** The actual lease time, in which the client has to renew its subscriptions to prevent the subscription and events to be deleted.
- **bufferSize** Actual size of the allocated event ring buffer.

Please note that the MAP may decide to enforce a different leaseTime and bufferSize as requested by the client. Thus, a client has to adhere to the leaseTime contained in the response and cannot assume that the originally requested time is going to be used for this subscription.

A subscription may not be accepted due to three main reasons. Either the panel is not able to accommodate more subscriptions (e.g. maximum number of subscribers exceeded), because

the subscription request violates access rights or because the subscription request is malformed.

In case the panel is rejecting the subscription due to resource limitations, a 503 “Service Unavailable” error will be send as a response to a request. In this case, a client may try to subscribe again at a later point in time or clean up existing subscriptions, if any.

In case the data in the request does not follow the specified scheme, a 400 ("Bad Request") will be returned by the MAP.

Request

Verb	*POST*
Query Parameters	(none)

Request Body:

Name	Type	Value Range/Format	Description
@cmd	String	SUBSCRIBE	Fixed String to identify this command
bufferSize	Number	1 - 640	Requested Number of events that will be stored by the MAP without overwriting an entry. The actual buffer size is decided by the MAP and provided in the response.
leaseTime	Number	10-600	Maximum number of seconds between two subscription renewal activities i.e. time between two event fetching requests encoded as JSON number. No fractions supported. The actual lease time is decided by the MAP.
subscriptions	Array of touple of {Array of EventType, Array of urls}	EventType is one of the following value CHANGED, CREATED, DELETED urls as defined in this specification	An array of subscriptions containing the following information: - EventType: Array of event types (CHANGED, CREATED, DELETED) - urls: List of links relative to the server base URL

			from which events are to be received. The list may contain strings that follow the string matching as defined in Chap. 6 String matching to reference multiple resources in a compact way.

Response (Success)

Return Code	201
Content	See below

Response Body

Name	Type	Value Range/Format	Description
bufferSize	Number		Actual, allocated buffer size
leaseTime	Number		Actual lease time
subscriptionURL	String		A URL to the location of the newly created subscription.

Response (Error)

See Chap. 11 Standard Errors.

```
{
        "bufferSize ": 4,
        "leaseTime": 600,
        "subscriptions": [
        { "eventType": ["CHANGE" "],
          "urls": [
              "/1.1.Area.2.5",
              "/1.1.Area.2.2"
            ],
         "allowed": true },
        { "eventType": ["CHANGED","CREATED", "DELETED"]
          "urls": [
              "/inc/*"
          ]
        }
        ]
}
```

Fig. 11 Subscription Request Example

5.3 Subscription Resource

The subscription resource represents an individual subscription. A client can inspect and delete its subscription at this resource. Events can be fetched from the resource by sending a POST.

Overview

OII Type	Supported Operations
OII.sub.1	Status Retrieval Fetch Events Unsubscribe

Object Structure

Name	Type	Value Range/Format	Description
@type	String	["OII.sub.1"]	Fixed type identifier
@self	String	URL starting with "/"	Link to the current resource
leaseTime	Number		The actual lease time, in which the client has to renew its subscriptions to prevent the

			subscription and events to be deleted. Minimum is 10. Maximum is 600.
bufferSize	Number		Actual size of the allocated event ring buffer.
subscriptions	Array of touple of {Array of EventType, Array of urls}	EventType is one of the following value CHANGED, CREATED, DELETED URLs encoded as String	Detailed information about the description in the same format as given during the subscription request

Example

```
{
  "@type": ["OII.sub.1"],
  "@self": "/sub/1",
  "leaseTime": 600,
  "bufferSize": 50,
  "subscriptions": [
   {
    "urls": [ "*" ],
    "eventType": [
     "CREATED",
     "DELETED",
     "CHANGED"
    ]
   }
  ]
}
```

5.3.1 Status Retrieval

Request

Verb	*GET*
Query Parameters	(none)

Response (Success)

Return Code	200
Content	Subscription object structure

Response (Error)

See Chap. 11 Standard Errors.

5.3.2 Fetch Events

A client fetches events from the buffer of this subscription by using a POST request with the defined, optional parameters in the body of the request. Once events are successfully delivered to the client, they will be deleted from the internal event buffer. Thus, events can only be fetched once.

Fetching the events is done by sending a POST request. Parameters are defined to allow a fine grained control on the event delivery to the client. The available parameters are:

- **maxEvents** The maximum number of events contained in the answer to this request. If maxEvents is omitted, the number of events is set to the bufferSize of the subscription. maxEvents is used to assure that the client only fetches as much events as it can process in one batch.
- **minEvents** The minimum number of events contained in the answer to this request, i.e. a request will return when at least minEvents are available. If minEvents is omitted or minEvents is specified as 0, minEvents has the same value as maxEvents (at maximum the bufferSize of the subscription).
- **maxTime**The maximum time in seconds a request will block. When maxTime expires, the panel will reply with the currently available events. If no events are available, an empty event list will be provided. The default value of maxTime is 0. Thus, if maxTime is omitted or set to 0 the request will return immediately with the available events (up to maxEvents). The maximum possible value for maxTime is 100. Thus, the call will never block longer than for 100 seconds.

The panel will respond to a POST request as soon as the first of the previously specified conditions is fulfilled. Thus, in case minEvents is not reached, the answer will return after maxTime. In case minEvents are reached, the response is send immediately.

By adjusting maxEvents, minEvents and maxTime the client has the opportunity to optimize the poling behaviour to its need. For example, when the client expects events rarely (e.g. alarm messages) it could set “minEvents”: 1 and “maxTime”: 100. Thereby, the client is notified as soon as a single event comes in. Using a long maxTime ensures that the poll request does not need to be repeated often. Similarly, if it is expected that many events come in (e.g. state changes in a disarmed area), the throughput can be improved by choosing a large minEvents number e.g. “minEvents”: 100 to make sure that the data is transmitted efficiently. maxEvents can be defined to ensure that not too many events are fetched which may not be possible to be handled be the client e.g “maxEvents”:200.

In addition, a client can extend its lease by sending a request with “maxEvents” set to 0. Thereby, the lease is extended and the response does not contain any events. This is particularly useful, if the client is currently not able to process any events but wants to keep its subscription on the panel.

Request

Verb	*POST*
Query Parameters	(none)

Object Structure

Name	Type	Value Range/Format	Description
@cmd	String	FETCHEVENTS	Fixed String to identify this command
maxEvents	Number	Positive int. Values beyond the allocated buffer size will have no effect on the request.	Optional. Default value is “unlimited”.
minEvents	Number	Positive int.	Optional. If not specified or specified as 0, minEvents = maxEvents. If a request specifies minEvents bigger than maxEvents, minEvents will be set to maxEvents.
maxTime	Number	0-100	Optional. Maximum number of seconds the call will block. Maximum value is 100. Minimum is 0.

Response (Success)

Return Code	200
Content	evt Object

Object Structure

Name	Type	Value Range/Format	Description
evts	Array of events		List of fetched events. May be empty, if no events are present or maxEvents = 0

Event entry structure

Name	Type	Value Range/Format	Description
id	String	<PanelEventID>.<SubscriptionSequenceNumber>	Id of this event consisting of panel and local subscriber id.
time	String	OII date time with millisecond precision	Local date time including time zone information for when the event occurred.
type	String	“CREATED”|”DELETED”|”CHANGED”	Event type classifier
props	Array	Array of Strings	A list of property keys whose value has changed. The list contains at least a single entry for an event of type CHANGED. In all other cases the list will be empty.
evt	Object		The resource representation as would be provided using a GET on that resource directly after the event happened.

Response (Error)

See Chap. 11 Standard Errors.

```
{
  "evts": [
        "id": 2345.4,
        "time": "2014-10-29T13:25:22.357-07:00",
        "type": "CHANGED",
        "props": [“armed”],
        "evt": {
           "@type": "IN.area.1",
           "@self": "/1.1.Area.2.1",
           "armed": false
         },
        "id": 4783.5,
        "time": "2014-10-29T13:29:01.25-07:00",
        "type": "CHANGED",
        "props": [“armed”],
        "evt": {
           "@type": "IN.area.1",
           "@self": "/1.1.Area.2.2",
           "armed": true
         }
  ]
}
```


Fig. 12 Example Fetch Events Response

5.4 Unsubscribe

This operation cancels a subscription. The MAP will free the event buffer associated to this subscription.

Request

Verb	*DELETE*
Query Parameters	(none)

Request Body:

-

Response (Success)

Return Code	204
Content	-

Response (Error)

See Chap. 11 Standard Errors.

6 String matching

In some cases a request from a client may need to reference multiple resources. This can happen for example when a list resource is accessed but not all entries are required. Also for event registration a mechanism is needed to specify the resources to which the client wants to subscribe to.

To this end, a string matching language is required that allows referencing of multiple OII.ids in an efficient way. In general, regular expressions are used for such use cases. However, regular expressions are deemed to be complex for the scope required in the context of the OII. In addition, the string matching is also to be used as part of query elements in a URL. Thus, the characters for the string matching language need to comply with the rules defined for HTTP URLs. Regular expressions do not follow these rules, so that they would need to be %-encoded which makes them not readable for humans. As a consequence, a simple string matching language is defined to cover most use cases required in the context of the OII.

Two main elements are defined for string matching:

- **Wildcard "*":** A wildcard matches zero or more characters and numbers. Thus, it follows the commonly known syntax as used in file systems where for example "/1.1.Area.2.*" matches all ids which start with "/1.1.Area.2.".
- **List of Numbers "( )":** A comma separated list of numbers or number range identifiers enclosed in round brackets i.e. "(",")". For example, it can be used to specify a simple list of numbers like "/1.1.Area.2.(2,5,8)" which matches "/1.1.Area.2.2", "/1.1.Area.2.5", "/1.1.Area.2.8". In addition, number ranges can be specified (e.g. 5-10, 100-200). A string matches the expression, if it contains a number out of the given interval including the given limits. For example, "/1.1.Area.2.(2-4)" matches "/1.1.Area.2.2", "/1.1.Area.2.3" and "/1.1.Area.2.4". The list may contain numbers and number ranges at the same time. For example, "/1.1.Area.2.(2-4,8)" matches "/1.1.Area.2.2", "/1.1.Area.2.3", "/1.1.Area.2.4", "/1.1.Area.2.8". Please note that "*" cannot be used as a replacement for a number in a list of numbers, as its meaning would be ambiguous.

In order for the string matching language to be used in the context of the OII, the characters "*","(" and ")" are explicitly reserved and may not be used as part of a OII resource url.

7 Feature discovery

In order for a client to interact with an OII server, the client needs to discover URLs of the relevant resources, such as "areas" or "points". For each version of the OII, the locations of these resources will be fixed. However, in the future the URLs may change. In order to facilitate compatibility, a client can discover the URLs of the main resources using a dedicated description resource.

To this end, any OII server MUST host a resource at the URL "/desc" directly at its root. This description resource contains links to all major, relevant resources (see below). The description should be the first starting point for any client that wants to interact with a device over OII.

Overview

OII Type	Supported Operations
OII.desc.1	Status Retrieval

Object Structure

Name	Type	Value Range/Format	Description
@type	Array of strings	[OII.desc.1]	Fixed string identifier
@self	String	"/desc"	Link to this resource
udn	String	[5digits].[11digits]	Serial number of the panel e.g. "81123.74325266505"
baseURL	String		Absolute URL to the OII Server described by this resource (e.g. https://myserver or https://10.24.2.75:586/server1)
friendlyName	String		Short description for the end user, e.g. "Intrusion Panel – Room 127"
firmwareVersion	String		The version of the firmware running on the device in format <Major>.<Minor>.<Micro> e.g. 1.3.9304
modelName	String		Identification of the MAP500 model, such as MAP5000, MAP5000-S, MAP5000-COM, MAP5000-SC
profiles	Array of Strings		It contains one or more Device Profile Identifiers (e.g. "IN.MAP5000.1.0", "IN.MAP5000.2.2"). A Device Profile Identifier references a dedicated MAP5000 resource model. In the future, a MAP can provide information about the set of resource models that are implemented. This

			allows a client to deduct the overall compatibility with the panel.
mainResources	Array of @type and ref tuple		A list of types and links for all main resources. The links are relative links. The absolute link can be created by prepending the baseURL to the given relative link of the resource.

Example Description Resource:


```
{
  "@self": "/desc",
  "@type": [ "OII.desc.1" ],
  "baseURL":  "https://192.168.1.20" ,
  "udn": "OIIMockup-1234567890",
  "friendlyName": "MAP5000 OII Test Server",
  "firmwareVersion": "1.42.99",
  "modelName": "ICP-MAP5000-S",
  "profiles": ["IN.MAP5000.1.0"],
  "mainResources": [
    {
      "@type": ["OII.list.1", "IN.areaList.1" ],
      "ref": "/areas"
    },
    {
      "@type": ["OII.list.1", "IN.pointList.1" ],
      "ref ": " /points"
    },
    {
      "@type": [ "OII.list.1", "OII.subList.1" ],
      "ref ": " /sub"
    },
    {
      "@type": [ "OII.list.1", "IN.incList.1"],
      "ref ": " /inc"
    },
    {
      "@type": [ "IN.history.1"],
      "ref": " /history"
    },
    {
      "@type": [ "IN.time.1"],
      "ref": " /time"
    },
    {
      "@type": [ "IN.userList.1"],
      "ref": " /users"
    }
  ]
}
```

7.1 Status Retrieval

Request

Verb	*GET*
Query Parameters	(none)

Response (Success)

Return Code	200
Content	Description object structure

Response (Error)

See Chap. 11 Standard Errors.

8 Supervision

In many cases, supervision of the MAP is required to indicate potential problems in the system. Thus, supervision is also required for communication over the OII. The TCP connection cannot be used for supervision purposes as a client may open and close multiple TCP connections over time. Thus, MAP supports supervision on a user basis. A connection to a user will be considered "available" if the MAP receives any OII message (get status, poll event, etc.) in a defined, configurable timespan e.g. 10 seconds. If no OII message is received in this interval, the connection is considered to be broken and a trouble will be detected. If supervision of redundant paths is required, each redundant client will need to use a different user for supervision, so that failure of a connection to each client can be detected. If both clients would use the same user, the panel would not recognize the loss of one connection, as the supervision timer would be reset by the OII message from the other, working client. Thus, the same user may be used, if only a trouble shall be generated if both clients are unavailable.

Every OII message results in a response to a client, thus the proposed solution will also be sufficient for the client to decide whether the MAP is still available.

Supervision is a feature that has to be configured with the MAP over RPS and cannot be configured over the OII. As any request from a user will be used to reset the supervision timers, no dedicated supervision resource is required.

9 Discovery

Auto discovery of a MAP is currently not supported. Clients need to know the IP address of the panel to use the OII. In case DNS is available, the serial number of the panel can be used as a domain name where the "." Is replaced by "-" (e.g. 94047.24269997507 results in 94047-24269997507 as a domain name). Future releases may include a discovery feature, allowing discovery of available panels in the network.

10 Communication

This chapter contains details on the communication protocol properties.

10.1 TCP

A client may use one or many TCP connections to issue requests to the MAP. A client is also free to either open a new connection for a new request or reuse an already established connection. The maximum number of TCP connection to the MAP is 20.

10.2 URI Encoding

URI and thereby @self must be conformant to RFC3986 [URI]. Furthermore, the characters "*","(" and ")" are explicitly reserved and may not be used as part of a URI and @self, in order to enable use of the string matching language of the OII.

10.3 Responses and Content Types

All resources in the scope of the OII base specification will be in JSON. Thus, the Content-Type "application/json" is expected for responses with Code 200/201. The 409 error code is used when there are application reasons that prevent the execution of a request (e.g. trying to arm an area that is not ready to arm).

The following list provides an overview of the return codes used in the scope of the OII and which content and content type they will provide. The individual resources defined in the OII resource model will follow this definition.

Return Code	*Content Type*	*Content*
200 "OK"	application/json	Object structure as defined in resource model
201 "Created"	application/json	Object structure as defined in resource model
202 "Accepted"	-	None
204 "No Content"	-	None
400 "Bad Request"	-	None
403 "Forbidden"	-	None
404 "File not Found"	-	None
409 "Conflict"	text/plain	String identifier explaining the reason for the conflict
503 "Service Unavailable"	-	None

In the current version of the OII content negotiation is not supported. Thus, the client has to accept the content format as defined.

10.4 HTTP Response Codes

The following HTTP response codes are those considered to be most relevant for the OII.

10.4.1 200 ("OK")

This response code indicates a successful transaction. It will be returned for a successful GET request. The response will contain the resource representation. PUT, POST or DELETE will

return 201 ("Created") after successful execution or 202 after successful acceptance or 204 (“No Content”) after successful execution.

10.4.2 201 ("Created")

This response code indicates that a new resource has been created. This can happen as a result to a PUT or POST request.

10.4.3 202 (“Accepted”)

This response code indicates that the request has been accepted but the processing has not been completed. The request may or may not eventually succeed.

10.4.4 204 ("No Content")

This response code indicates a successful transaction, which does not contain any additional data in the response. This will be used, for example. as response to a successful DELETE.

10.4.5 400 ("Bad Request")

This response code indicates a malformed or otherwise faulty request.

10.4.6 401 ("Unauthorized")

This response code indicates that the client does not have the appropriate access rights to execute the requested action on the server. It indicates that an authorization needs to be done for the request.

10.4.7 403 ("Forbidden")

A valid request was sent, but the user is not allowed to conduct the requested operation.

10.4.8 404 ("Not Found")

This response code indicates that the referenced resource does not exist.

10.4.9 409 (“Conflict”)

This command code is returned when a command is not executed due to application specific reasons. The body of the error response will contain further information on why the command was not executed.

This response code is also returned when a command on a list resource was issued with an “atomic” parameter. The code indicates that the execution of the command was not possible. The body of the response will contain the list of resource URLs which prevented execution of the command.

10.4.10 414 (“Request-URI Too Long”)

Response code is used if the URI exceeds the maximum supported size (128 bytes). In the context of the OII are intended to be kept short, but a client may increase the size of a URI by adding query parameters. String matching can be used to reduce the query length.

10.4.11 503 ("Service Unavailable")

This response code indicates that the server is in a temporary overload condition and thus unable to serve the request. The client can retry the request at a later point in time

10.5 Security

The OII provides two levels of communication security. On transport level, TLS is used. Particular focus is given on only supporting TLS in a configuration that is compliant with the BSI guidelines [BSI]. In addition, each request to the OII is authorized using username and password information. The same access rights are granted to a user over the OII as over a MAP key pad (please note that a user has to be configured as OII user via RPS). This ensures that the authorization is handled consistently throughout the MAP system.

10.5.1 Transport Layer Security

The OII supports TLSv1.2 with the following cipher suites with forward secrecy:

TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA256
TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256
TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA384
TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384
TLS_DHE_RSA_WITH_AES_128_CBC_SHA256
TLS_DHE_RSA_WITH_AES_128_GCM_SHA256
TLS_DHE_RSA_WITH_AES_256_CBC_SHA256
TLS_DHE_RSA_WITH_AES_256_GCM_SHA384

The following cipher suites are supported in addition to the previously listed once if perfect forward secrecy is not required (RPS configuration options):

TLS_ECDH_RSA_WITH_AES_128_CBC_SHA256
TLS_ECDH_RSA_WITH_AES_128_GCM_SHA256
TLS_ECDH_RSA_WITH_AES_256_CBC_SHA384
TLS_ECDH_RSA_WITH_AES_256_GCM_SHA384

This follows the suggestions of the BSI in [BSI]. The OII by default will not support ciphers other than proposed by the BSI in order to ensure state of the art security and to prevent adding vulnerabilities by usage of out dated ciphers.

For clients that cannot support TLSv1.2, the OII shall allow downgrade to TLSv1.0. With TLSv1.0, the following cipher suites will be supported:

TLS_RSA_WITH_RC4_128_MD5
TLS_RSA_WITH_RC4_128_SHA
TLS_RSA_WITH_AES_128_CBC_SHA
TLS_RSA_WITH_AES_256_CBC_SHA
TLS_DHE_RSA_WITH_AES_128_CBC_SHA

The downgrade to TLSv1.0 can be activated using RPS configuration options. Downgrade to TLSv1.0 is possible only if perfect forward secrecy is not required.

The MAP is not going to validate client certificates. Thus, similar as with banking portals, any client is allowed to connect to the MAP via TLS, but any activity on the OII will involve application layer security measures (username, password). The MAP itself will provide a self-signed certificate which is unique for each MAP product.

10.5.2 Certificate

Each MAP5000 has a unique, self-signed RSA certificate. The public part of the certificate can be downloaded as part of the TLS handshake. A client that wants to do sever authentication has to gain access to the certificate in a secure manner (e.g. directly connecting to the MAP5000 via the build in Ethernet plug or via a secure network) to ensure that the certificate belongs to the desired MAP. The upload of certificates to the MAP5000 is currently not possible.

10.5.3 Application Layer Security

OII requires HTTP authentication. HTTP Digest is used to provide username and password information. Username and password are matched with the user database as configured via RPS. An OII request will only be executed if the password is valid and the user has the appropriate access rights to conduct the operation. This even includes inspecting the status of an area. Thus, an OII user will not be able to see or execute functionality that would not appear on a keypad when the user is logged in.

In the current version of the OII, only users that have all permissions in the system can be configured as OII users.

11 Standard Errors

The following list contains the standard errors and their semantics. Any client that uses the OII is expected to support at least the given errors.

These errors are:

Code	Description	Meaning
400	Bad Request	Request content or filter not conformant to the specification.
401	Unauthorized	User credentials are required to execute the request. This error occurs when no or incorrect credentials are provided.
403	Forbidden	User not allowed to execute the command or the command not supported by the resource. With reactionless interface activated, all commands are forbidden.
404	File not Found	Resource does not exist or user does not have permissions to access the resource.
409	Conflict	Application specific reasons prevent command execution e.g. arm on an area that is not ready to arm.
503	Service	Server cannot serve the request at the moment. The client can try

	unavailable	to execute the same request at a later point in time.

12 Change History

Date	Description	Author
17.04.2014	First version for external review	Barisic
07.05.2014	Changed Event timestamp, Changed query parameters names from "_" to "-"	Barisic
27.08.2014	Removed OII.Manufacturer from object structure of /desc (Section 8)	Arun Ram
19.09.2014	Clean up	Barisic
16.10.2014	Section 9 Feature Discovery – removed OII.Manufacturer from the example	Arun Ram
30.10.2014	Introduced BaseURL for description Defined property naming concept Renamed OII.Type and OII.ID to @type and @self Introduced @cmd Removed hierarchical property keys Defined timestamp format Changed filter from id to url	Barisic
21.11.2014	Corrected incident url in subscription example in Fig.12	Arun Ram Moorthi
12.12.2015	Added description on TCP timeout behaviour Added MAX time definition Added date definition	Daniel Barisic
14.12.2015	Added atomic parameter for list resource Added allowed for subscription Added subscription description for list resources	Daniel Barisic
09.01.2015	Section 5.2.1 Subscription List Resource – Status Retrieval : Corrected "ri" to "url" Section 11.5.10 : 409 ("Conflict") : Updated description Section 12 : Modified descriptions for 403 "Forbidden" and 409 "Conflict" error codes.	Arun Ram Moorthi
18.01.2015	Section 8 : Feature Discovery : Object Structure - Modified @type to [OII.desc.1]	Arun Ram Moorthi
18.03.2015	Added section 3.8 Command Execution Section 4 List Resource: Removed "allowed" flag as query parameter. Updated error responses for operations on lists when atomic true/false. Added 403 response when unsupported operation requested on a list resource	Daniel Barisic Arun Ram Moorthi

	Section 5.2.1 : Removed allowed flag Section 5.2.2 : Removed allowed flag Section 5.3.2 : Updated description for minEvents parameter Removed Section 7 : Caching Added Section 10.4 : Responses and Content Types Added Section 10.5 : Added 202 (“Accepted”) Removed 304 (“Modified”), 405(“Method not allowed”), 412 (“Precondition Failed”) Updated Section 10.6.3 : Application Security Updated Section 11 : Standard Errors : Description updated	
26.03.2015	Minor, editorial clean up	Barisic
21.07.2015	Correction of return code statement in Section 4	Barisic
06.08.2015	Removed DH based, non perfect forward secrecy ciphers.	Barisic
01.09.2015	Added comment on FetchEvents with regards to minEvents and maxEvents in relation to the maxBufferSize of the subscription	Schröer
20.11.2015	Section 10.5.1 : Updated section with support for TLSv1.0 ciphers	Arun Ram Moorthi

13 References

[HTTP]	http://www.ietf.org/rfc/rfc2616.txt
[URL]	http://www.ietf.org/rfc/rfc3986.txt. HTTP spec. references http://www.ietf.org/rfc/rfc2396.txt but RFC3986 obsoletes RFC2396.
[JSON]	Specification http://www.ecma-international.org/publications/files/ECMA-ST/ECMA-404.pdf, Official Website http://www.json.org/
[JSTY]	JSON Style Guide https://google-styleguide.googlecode.com/svn/trunk/jsoncstyleguide.xml
[RFC3339]	IETF RFC3339 “Date and Time on the Internet: Timestamps”, http://www.ietf.org/rfc/rfc3339.txt
[BSI]	Bundesamt für Sicherheit in der Informationstechnik, Technische Richtlinie TR-02102-2 (Version 2014-01)

Bosch Sicherheitssysteme GmbH
Robert-Bosch-Ring 5
85630 Grasbrunn
Germany
www.boschsecurity.co
m